

Sentry: Separating Policy Admission from Predictive Microservice Placement

Anonymous Author(s)

Abstract

A microservice scheduler must sometimes reject a lightly loaded node because the service is not permitted to run there. Among permitted nodes, it must then decide whether to retain the current placement or move the service as resource pressure changes. We present SENTRY, a scheduler that separates policy admission, predicted node risk, destination reuse, and an empirical headroom check. It retains an admissible source and otherwise searches a shortlist ordered by previous selections, with a final admission check before recording a destination. We evaluate the decision procedure using Alibaba’s 2021 microservice trace and synthetic policy attributes. On 11,629 held-out events across 12,727 candidate nodes, it requests migration for 39.118% of events, compared with 42.850% for a current-state learned baseline. The selected nodes’ recorded next-interval unsafe rate is 0.043%; this is an observational proxy, not an outcome of executed migrations. Across 30 policy overlays, all 462,480 decisions pass the configured admission checks. Mean in-memory processing time is 20.247 ms per natural-workload timestamp. The results identify a migration–risk trade-off within replay; they do not establish application latency, post-placement capacity, or production safety.

1 Introduction

A placement decision can be operationally attractive and still be unacceptable. A node with spare CPU may lie outside a service’s permitted jurisdiction, lack encryption support, or fail an isolation requirement. Once these restrictions are enforced, a second problem remains: a node that looks lightly loaded now may be approaching a resource boundary. Moving a service there can exchange one problematic destination for another.

This paper studies the decision procedure at that boundary between policy admission and resource management. We consider a controller that periodically reconsiders existing microservice placements. Its inputs are service requirements, node capabilities, and recent CPU and memory measurements. Its output is a decision to retain the current source, select a different node, or report that no candidate passes admission. The question is how to use temporal information without allowing a favorable resource score to override a mandatory policy condition.

Hard filtering before scoring is established practice. Kubernetes, for example, separates filtering, scoring, reservation, and binding [4]. We do not claim that this separation is new. Our focus is the interaction of source retention, a next-interval risk screen, destination-selection history, and

a separate headroom rule in repeated placement decisions. This is narrower than solving general edge–cloud orchestration: we do not optimize service-graph latency, network traffic, or migration execution.

SENTRY uses four checks with different purposes. A Boolean policy mask excludes prohibited nodes. A histogram gradient-boosting model uses recent node history to estimate whether CPU or memory will cross a fixed boundary in the next 30 seconds. Historical selection counts favor destinations that have been used less often. Finally, a persistence projection with empirical 95th-percentile residual margins checks headroom before a destination is recorded. The source is retained whenever it passes the policy, risk, and headroom conditions. Otherwise, the scheduler examines a bounded shortlist and expands the search if necessary.

The distinction between prediction and admission matters. A probability below the risk threshold is a model decision, not a safety guarantee. Likewise, passing a headroom projection does not establish that a node can absorb the incoming service: the projection does not add that service’s demand. These checks define the scheduler’s decision contract. We evaluate both whether it obeys that contract and whether selected nodes subsequently look safer in the unmodified trace.

Our evaluation replays CPU-burst events from Alibaba’s 2021 microservice trace. Policy attributes are synthetic because the trace does not provide them. On the natural test workload, SENTRY reduces migration decisions by 3.732 percentage points relative to a current-state learned baseline. Against a policy-and-headroom baseline, it instead requests 0.920 percentage points more migrations, while the recorded next-state unsafe rate falls from 0.155% to 0.043%. These are different trade-offs, not evidence that one scheduler dominates all alternatives. The replay does not execute migrations or modify future telemetry.

The paper makes three contributions:

- A placement decision procedure that combines source retention, hard policy admission, temporal risk screening, destination-use history, and a separate empirical headroom check.
- A replay comparison that distinguishes compliance with configured checks from observed next-state behavior, including current-state and component-removal controls.
- An analysis of shortlist cost, policy-overlay sensitivity, and the limits of conclusions drawn from unmodified trace replay.

2 Decision Model

At each timestamp t , separated by $\Delta = 30$ seconds, the scheduler receives an ordered batch $\mathcal{E}_t$ of events $e = (i, s, n_s, t)$ identifying an instance, its service, and its recorded source node. There are M candidate nodes in $\mathcal{N}$. Node CPU and memory observations are denoted $c_{n,t}$ and $m_{n,t}$; admission uses normalized node telemetry, not instance-level resource signals.

2.1 Policy and risk

Node n has jurisdiction j_n , trust rank τ_n , encryption capability η_n , and isolation capability ζ_n . Service s specifies permitted jurisdictions J_s and minimum capability values. Its policy predicate is

$$L(s, n) = \mathbb{I}[j_n \in J_s] \mathbb{I}[\tau_n \geq \tau_s^{\min}] \cdot \mathbb{I}[\eta_n \geq \eta_s^{\min}] \mathbb{I}[\zeta_n \geq \zeta_s^{\min}]. \quad (1)$$

The policy mask is an input to scheduling. The scheduler does not verify jurisdictions or produce hardware attestations; a deployment would need trusted mechanisms to supply and refresh these attributes.

The prediction target is a node's recorded threshold crossing:

$$y_{n,t+\Delta} = \mathbb{I}[c_{n,t+\Delta} > 0.8 \vee m_{n,t+\Delta} > 0.8]. \quad (2)$$

The learned score $p_{n,t} = f_\phi(\mathbf{h}_{n,t})$ estimates this target from observations ending at t . A node passes the risk screen when $p_{n,t} \leq \theta$. Threshold violations below refer to this configured screen, not to actual service-level objective violations.

2.2 Empirical headroom

For pre-test calibration transitions C , define the margins

$$q_x = Q_{0.95}\{|x_{n,t+\Delta} - x_{n,t}| : (n, t) \in C\}, \quad (3)$$

for $x \in \{c, m\}$. The headroom score is

$$g_{n,t} = \max\{(c_{n,t} + q_c)/0.8, (m_{n,t} + q_m)/0.8\}. \quad (4)$$

Admission requires $g_{n,t} \leq 1$. Q95 denotes the marginal empirical residual percentile; it is not a joint 95% CPU-and-memory guarantee. The calculation excludes incoming service demand, interference, and migration transients. We therefore call it a headroom check rather than a certificate of post-placement feasibility.

2.3 Batch state and admissibility

Let A_t contain every source node in the current event batch. Let $U_t(n)$ count new destination reservations during that batch, with limit C , and let $H(n)$ count earlier committed decisions across timestamps, including source retention. Counters are updated after each decision; U_t is reset at the next timestamp.

A source is retained exactly when $L(s, n_s) = 1$, $p_{n_s,t} \leq \theta$, and $g_{n_s,t} \leq 1$. Otherwise the base candidate set is

$$\mathcal{B}_e = \{n : L(s, n) = 1, n \notin A_t, U_t(n) < C\}. \quad (5)$$

The full feasible set additionally requires the risk and headroom checks:

$$\mathcal{F}_e = \{n \in \mathcal{B}_e : p_{n,t} \leq \theta, g_{n,t} \leq 1\}. \quad (6)$$

The reservation limit counts decisions; it does not reserve CPU or memory bytes. Excluding A_t avoids choosing another active source in the same batch. Neither rule models concurrent migration execution.

3 Scheduler Design

3.1 Prepare shared state once per timestamp

Policy masks are reusable while service requirements and node capabilities remain unchanged. For each timestamp, the scheduler constructs node features and batches model inference over the nodes required by the event batch. The resulting risk and headroom vectors are reused by individual events. Computing a risk score does not admit a node: policy remains a mandatory condition on every selected source or destination.

The reported predictor uses 22 features covering current CPU and memory, projected values, joint utilization, headroom, two preceding snapshots, changes, and short-window statistics. These features use three consecutive snapshots ending at t . The feature families describe the predictor's inputs but do not fully specify its implementation. The serialized feature schema, individual feature formulas, and model hyperparameters are needed for exact reproduction and are not included with this draft.

3.2 Shortlist before final screening

For a source that fails admission, SENTRY orders $\mathcal{B}_e$ lexicographically by $(H(n), g_{n,t}, \text{id}(n))$ and takes its first k nodes. It then applies the risk and headroom screens to this shortlist. If none pass, it screens the remaining base candidates. Among accepted nodes it selects the minimum

$$(H(n), p_{n,t}, g_{n,t}, \text{id}(n)). \quad (7)$$

Thus, historical use is the primary ranking key, and predicted risk breaks ties in that key. The first shortlist is drawn from the policy-and-batch candidate set, not from $\mathcal{F}_e$. This distinction matters: shortlisting before screening can choose a different node from ranking all feasible nodes. Equality with full ranking is an empirical result for the reported workloads, not a general property of the algorithm.

If all base candidates fail, the scheduler returns an infeasible decision. Search expansion guarantees that a candidate passing the fixed snapshot checks is not missed merely because it lies outside the initial shortlist. It does not guarantee that every event has a feasible destination.

3.3 Final check and state update

Algorithm 1 consolidates the decision path. Before recording a migration, the scheduler repeats the policy, risk,

Algorithm 1: Placement decisions for one times-tamp

Input: Ordered events $\mathcal{E}_t$; policy masks; node snapshots; history H
Prepare p_t, g_t once; form A_t ; set $U_t \leftarrow 0$;
foreach $e = (i, s, n_s, t) \in \mathcal{E}_t$ **do**
 $\pi(e) \leftarrow \emptyset$;
 if $L(s, n_s) = 1$ and $p_{n_s, t} \leq \theta$ and $g_{n_s, t} \leq 1$ **then**
 $\pi(e) \leftarrow n_s$; $H(n_s) \leftarrow H(n_s) + 1$;
 continue;
 Form $\mathcal{B}_e$ using Eq. (5);
 $K \leftarrow$ first k nodes by (H, g, id) ;
 $D \leftarrow \{n \in K : p_{n, t} \leq \theta, g_{n, t} \leq 1\}$;
 if $D = \emptyset$ **then**
 $D \leftarrow \{n \in \mathcal{B}_e \setminus K : p_{n, t} \leq \theta, g_{n, t} \leq 1\}$;
 if $D = \emptyset$ **then**
 continue;
 $n^* \leftarrow$ minimum of D by Eq. (7);
 Recheck all conditions in Eq. (6) for n^* ;
 if any recheck fails **then**
 continue;
 $U_t(n^*) \leftarrow U_t(n^*) + 1$;
 $H(n^*) \leftarrow H(n^*) + 1$; $\pi(e) \leftarrow n^*$;
return π ;

headroom, active-source, and reservation checks. A failed recheck cannot fall through to a committed decision. In the replay, telemetry is fixed within a batch and decisions are processed sequentially. A live controller would also need atomic resource reservation and a defined retry policy for changing state.

The decision procedure preserves its admission predicates under a fixed snapshot: retention tests all three source conditions, and migration is recorded only after every destination condition passes. This is a statement about the algorithm’s checks. It says nothing about the accuracy of the predictor, the truth of supplied policy attributes, or future resource use.

3.4 Cost

For R_t nodes requiring inference and a fixed trained model, feature construction and inference take $O(R_t)$ work. Building a candidate set is $O(M)$ per migrating event. A heap-based top- k implementation can select the shortlist in $O(M \log k)$, then rank accepted candidates in $O(k)$; expansion adds an $O(M)$ scan. A full sort instead costs $O(M \log M)$. These are implementation bounds; timing results alone do not establish which selection primitive was used. State requires $O(M)$ space plus $O(PM)$ for P policy masks. Shortlisting reduces candidate-ranking work, not

the number of nodes whose features necessarily need to be constructed.

4 Evaluation Method

We ask whether temporal history changes migration decisions, what each admission component contributes, how destinations are distributed, and how much computation the decision procedure requires. We compare the complete procedure with component ablations under the same trace-replay protocol.

4.1 Trace and event workload

The study uses Alibaba Microservices Trace 2021 [1, 5], combining Node_0 with MSResource_0. The full node archive contains 18,868,400 records, 13,118 nodes, and 1,440 timestamps at 30-second intervals. Removing incomplete full trajectories gives 13,058 nodes. Local alignment with the clean microservice shard yields 12,727 nodes over 120 timestamps and 1,527,240 node observations. The microservice shard contains 11,561,072 clean records, 1,303 services, and 96,346 service-instance pairs.

Instance CPU signals are used to trigger decisions, not to infer node capacity. After a 300-second warm-up, an event is emitted when the instance CPU signal increases by more than 0.05 over one interval. The rule produces 20,419 events; excluding the last timestamp, whose outcome is unavailable, leaves 20,324 events over 108 timestamps. This is a constructed workload of placement reconsiderations, not a trace of new service arrivals.

The natural workload preserves the observed service-frequency imbalance: its ten most frequent services account for approximately 46% of events. A second workload retains at most two events per service per timestamp, using deterministic selection. It contains 6,875 events. The balanced test subset has a top-ten service share of 22.55%. Both test workloads use the same 59 timestamps, so they are sensitivity views of the same trace rather than independent replications.

4.2 Information boundary and model selection

An event at t uses the instance change ending at t . Its decision uses node observations at $t, t - \Delta, t - 2\Delta$; evaluation reads the selected node’s recorded state at $t + \Delta$. The replay leaves that future state unchanged. A later event still refers to the source recorded in the trace; the evaluation is not a closed-loop simulation of the resulting placements.

The chronological split is at 1,800,000 ms. The reported partitions are shown in Table 1. Model fitting uses development data; estimator selection and operating-point selection use the pre-test region. The final test region is reserved for evaluation. Exact split inequalities, label-boundary handling, and the provenance of projection features used during fitting must be recovered from the experiment code to audit this information boundary fully.

Table 1. Reported chronological partitions.

Partition	Transitions	Natural events
Development	432,718	–
Validation	190,905	–
Pre-test total	623,623	8,695
Test	750,893	11,629

The balanced pre-test and test subsets contain 3,088 and 3,787 events. The principal model is scikit-learn histogram gradient boosting [8]. Its direct predictive control uses current telemetry without temporal lags or changes. Both use the same chronological partitions. Reported model metrics include ROC-AUC, PR-AUC, Brier score, log loss, and expected calibration error. A separate probability-calibration transform is not specified; a low calibration error alone does not identify such a transform.

The temporal threshold is 0.011362695354780319; the current-state threshold is 0.004292615073088162. These were selected at matched pre-test operating points. A stricter temporal threshold, 0.0020594307448572016, is used for sensitivity analysis. The empirical headroom margins are

$$q_c = 0.025929729975254, \quad (8)$$

$$q_m = 0.00021962105567108203. \quad (9)$$

The operating boundary is 0.8 for both resources. Deterministic headroom cutoffs of 1, 0.99, and 0.975 provide comparison points. The default shortlist size is $k = 256$, compared with 128, 512, 1024, 2048, and full ranking. Reservation capacity is $C = 1$, with $C = 2$ as a sensitivity check. Settings were selected before final testing; the selection logs are not included with this draft.

4.3 Synthetic policy attributes

The trace has no jurisdiction, trust, encryption, or isolation labels. The evaluation assigns them using a stable hash of identifiers and a policy seed, independently of resource telemetry. Nodes receive one of four jurisdictions, one of three trust ranks, and Boolean encryption and trusted-execution attributes. Services receive one of six policy classes. Under the reference seed, legal-node counts are 12,727 (global public), 10,320 (global protected), 6,206 (EU restricted), 2,613 (US restricted), 1,501 (APAC restricted), and 866 (EU confidential).

Thirty deterministic overlays vary node capabilities and service requirements while retaining the trace and model. This tests behavior under the chosen policy generator, not compliance with actual Alibaba policies. In particular, the native source baseline can violate an overlay because that overlay was assigned after collection. These violations do not describe a production compliance failure.

4.4 Controls and ablations

All methods share events, event order, telemetry, and fixed tie breaking. Applicable methods also share the policy overlay, active-source exclusion, and reservation rules. Each method retains its source if that source passes its own admission rule. Consequently, comparisons measure both retention and destination choice.

- *Observed source* retains the trace source without admission.
- *Random-policy-Q95* and *Least-loaded-policy-Q95* enforce policy and headroom, then choose a random or minimally utilized node. Random selection is repeated with 20 deterministic seeds.
- *Policy-only-fair* uses policy and selection history. *Policy-Q95-fair* adds headroom; *Policy-boundary-0.99* uses a stricter headroom cutoff.
- *Policy-current-HGB-Q95* replaces temporal features with current-state learned risk.
- *Temporal-Q95-no-policy* removes the policy gate; *Policy-temporal-no-projection* removes the headroom check.

Together with SENTRY, these are ten methods, including the observed-source control. They are internal controls, not implementations of published orchestrators. No direct performance comparison with Kubernetes, NetMARKS, or another deployed scheduler is claimed.

4.5 Metrics and uncertainty

Migration rate counts decisions whose selected node differs from the recorded source. Policy, temporal, and Q95 violation rates count failures of $L(s, n) = 1$, $p_{n,t} \leq \theta$, and $g_{n,t} \leq 1$, respectively. The temporal metric applies SENTRY’s temporal screen to all methods, including those optimized for another admission rule. It measures agreement with that screen and is not an independent measure of predictive quality.

The native next-state unsafe rate averages $y_{n,t+\Delta}$ for selected nodes. It describes their unmodified recorded trajectories. It does not measure overload caused or prevented by migration. Search expansion and infeasible-event counts are reported separately; no infeasible decisions are reported in the evaluated configurations.

For migrated-destination counts z_n , Jain’s index is

$$J = \frac{(\sum_n z_n)^2}{M \sum_n z_n^2}. \quad (10)$$

It includes nodes with zero assignments and measures destination spread, not fairness among tenants or services. For the no-migration control it is undefined; the tables mark it accordingly. Top-ten share is the fraction of migrations assigned to the ten most-used nodes.

The paired timestamp-cluster bootstrap samples the 59 test timestamps with replacement and includes every event in each sampled timestamp. The same samples are applied to both methods. Random-baseline replicate identity is also

Table 2. Node-risk prediction on the final test transitions.

Metric	Current-state HGB	Temporal HGB
ROC-AUC $\uparrow$	0.996944	0.998248
PR-AUC $\uparrow$	0.984894	0.990308
Brier score $\downarrow$	0.009858	0.008578

sampled where applicable. Reported intervals are percentile 95% intervals from 20,000 resamples of the difference SENTRY minus baseline. This preserves within-timestamp dependence but does not preserve correlation between adjacent timestamps or propagate uncertainty from fitting a different risk model.

4.6 Timing and artifact status

One latency sample is recorded per test timestamp. The in-memory path includes feature construction, all required model calls, filtering, shortlisting, ranking, reservations, and final checks. Telemetry collection, network and file I/O, serialization, binding, and migration execution are excluded. The reported platform is a Dell Precision 3561 running Ubuntu under WSL2 with Python 3.12.3. The exact CPU, memory allocation, thread settings, warm-up protocol, and repeat count are not specified, limiting reproducibility of the absolute timing results.

The available package contains manuscript sources and rendered figures. It does not contain the preprocessing code, model, policy generator, replay implementation, environment lockfile, or raw measurements needed to reproduce these results. Artifact availability should therefore not be inferred from the public availability of the source trace.

5 Results

5.1 Temporal node-risk prediction

We first examine whether short temporal histories improve the prediction of next-interval node risk. Table 2 compares the temporal histogram-gradient-boosting model with its current-state counterpart on the final held-out node transitions.

Both models achieve high discrimination, but the temporal representation improves every common test metric. The increase in PR-AUC is particularly relevant because unsafe transitions form the minority class. The lower Brier score further shows that temporal history improves the quality of the predicted probabilities, although it does not identify a separate probability-calibration procedure. For the temporal model, the final test log loss is 0.030480 and the expected calibration error is 0.000970. Table 3 compares the learned temporal screens with deterministic guardrail rules at closely matched acceptance levels. The default temporal operating point accepts 82.35% of node transitions, which is close to the 82.11% acceptance of the Q95 rule. Its unsafe rate

among accepted nodes is 0.0571%, compared with 0.1343% for Q95, while retaining a slightly larger share of actually safe nodes. The same pattern appears at the stricter operating point: temporal screening achieves a lower unsafe rate than the deterministic 0.99 boundary at nearly identical acceptance.

These results establish that temporal history contributes information that is not captured by current utilization or a fixed projected-headroom rule. The default temporal threshold is therefore used in the complete scheduler, while the deterministic Q95 rule remains as the final pre-commit verification stage.

5.2 Replay comparison on the natural workload

Table 4 presents the main comparison on the natural test workload of 11,629 events. The observed-source control illustrates the difficulty of retaining the native placements: 25.875% of events remain on policy-inadmissible sources, 19.512% exceed the temporal-risk threshold, and 19.451% fail the Q95 condition. Enforcing policy alone removes policy violations but leaves substantial operational exposure. Policy-only-fair records temporal and Q95 violation rates of 13.011% and 13.019%, respectively. Conversely, removing policy from the temporal scheduler preserves low operational risk but leaves 25.411% policy violations. These two controls show that compliance and operational suitability address different requirements and neither can replace the other.

The comparison with the current-state learned scheduler changes the feature set and the associated admission threshold. Current-HGB migrates 42.850% of events and leaves a temporal-violation rate of 0.370%. SENTRY reduces migration to 39.118%, removes the remaining temporal violations, and reduces the next-state unsafe rate from 0.069% to 0.043%. The comparison with Policy-Q95-fair isolates the contribution of learned temporal screening. SENTRY increases migration by only 0.920 percentage points, while reducing the temporal-violation rate from 0.980% to zero and the next-state unsafe rate from 0.155% to 0.043%. Finally, the no-projection ablation isolates the pre-commit guardrail. No projection already eliminates policy and temporal violations, but 1.144% of its final selections fail the Q95 condition. Adding the pre-commit projection increases migration by 0.929 percentage points and removes these violations without increasing the next-state unsafe rate. The least-loaded baseline also demonstrates the importance of fairness-aware selection. Although it satisfies policy and Q95 conditions, its migrated-destination Jain index is only 0.009 and its ten most-used nodes receive 13.282% of all migrations. Under the same policy and Q95 conditions, Policy-Q95-fair increases Jain fairness to 0.291 and reduces the top-ten share to 0.450%.

Table 3. Test-set operating-point comparison. Unsafe rate is measured among accepted node transitions; safe retention is the fraction of actually safe transitions retained by the rule.

Screen	Threshold or cutoff	Acceptance (%)	Unsafe among accepted (%)	Safe retention (%)
Temporal availability	0.011362695	82.349	0.0571	93.287
Q95 guardrail	1.000	82.106	0.1343	92.939
Temporal safety	0.002059431	75.714	0.0236	85.799
Boundary guardrail	0.990	75.819	0.0617	85.885
Strict Q95 guardrail	0.975	62.960	0.0311	71.340

Table 4. Replay results on the natural test workload (11,629 events). Rates and Top-10 are percentages; Jain is unitless. Risk means temporal-screen violation. Unsafe is the native next-state proxy. Short names refer to the controls in Section 4.

Method	Migr.	Policy	Risk	Q95	Unsafe	Jain	Top-10
Observed source	0.000	25.875	19.512	19.451	12.340	–	–
Random	38.198	0.000	1.392	0.000	0.190	0.191	1.353
Least-loaded	38.198	0.000	0.920	0.000	0.155	0.009	13.282
Policy-only-fair	25.875	0.000	13.011	13.019	7.215	0.189	0.665
Policy-Q95-fair	38.198	0.000	0.980	0.000	0.155	0.291	0.450
Boundary-0.99	43.323	0.000	0.404	0.000	0.095	0.332	0.397
Current-HGB	42.850	0.000	0.370	0.000	0.069	0.329	0.401
No policy	20.741	25.411	0.000	0.000	0.060	0.190	0.415
No projection	38.189	0.000	0.000	1.144	0.043	0.290	0.450
SENTRY	39.118	0.000	0.000	0.000	0.043	0.299	0.440

5.3 Service-balanced workload

The service-balanced workload tests whether the main observations depend on the event concentration of a small number of services. Table 5 reports the results for 3,787 test events.

The balanced workload reproduces the natural-workload pattern. SENTRY passes all configured policy, temporal-risk, and Q95 checks with a migration rate of 40.190% and a next-state unsafe rate of 0.106%. Current-HGB migrates more events, at 43.200%, while leaving 0.502% temporal violations. Policy-Q95-fair migrates slightly fewer events, at 38.791%, but its temporal-violation and next-state unsafe rates increase to 1.400% and 0.290%, respectively. The fairness comparison becomes even more pronounced in this workload. Least-loaded concentrates 39.959% of migrations on its ten most-used destinations and achieves a Jain index of only 0.002. Policy-Q95-fair reduces the top-ten share to 0.681% and raises the Jain index to 0.115. SENTRY retains a similarly distributed assignment pattern while adding temporal screening and pre-commit verification.

5.4 Shortlist-size sensitivity

Table 6 reports the candidate-selection microbenchmark for fairness-aware shortlists. Full fairness-aware candidate evaluation requires 72.930 ms on average. Restricting the initial candidate set sharply reduces this cost.

The $k = 256$ configuration reproduces the full fairness-aware selections, requires no search expansion, and preserves a minimum accepted-candidate slack of 130 nodes on the natural workload and 229 nodes on the balanced workload. Its isolated shortlist latency has a P95 of 5.937 ms and a P99 of 7.576 ms. The smaller $k = 128$ setting is faster, but its minimum natural-workload slack falls to two candidates. SENTRY therefore uses $k = 256$ as the default configuration.

5.5 Paired timestamp-cluster comparisons

Table 7 reports the principal paired timestamp-cluster bootstrap comparisons. Differences are calculated as SENTRY minus the corresponding baseline. For migration and violation rates, an interval below zero favours SENTRY.

Relative to the current-state learned scheduler, SENTRY achieves significantly lower migration and temporal-violation rates on both workloads. Relative to Policy-Q95-fair, it incurs a small and precisely estimated migration increase while significantly reducing temporal violations and the next-state unsafe rate. Relative to the no-projection ablation, the additional migration is accompanied by the complete removal of Q95 violations. The natural-workload next-state interval against Current-HGB includes zero; this comparison does not establish a strict improvement in that proxy at the reported confidence level. Temporal violations measure agreement with the temporal model, while the

Table 5. Replay results on the service-balanced test workload (3,787 events). Rates and Top-10 are percentages; Jain is unitless. Risk means temporal-screen violation.

Method	Migr.	Policy	Risk	Q95	Unsafe	Jain	Top-10
Observed source	0.000	24.611	21.917	21.310	13.890	–	–
Random	38.791	0.000	1.863	0.000	0.334	0.094	2.155
Least-loaded	38.791	0.000	1.400	0.000	0.290	0.002	39.959
Policy-only-fair	24.611	0.000	14.787	14.180	8.635	0.073	1.073
Policy-Q95-fair	38.791	0.000	1.400	0.000	0.290	0.115	0.681
Boundary-0.99	43.623	0.000	0.528	0.000	0.158	0.130	0.605
Current-HGB	43.200	0.000	0.502	0.000	0.158	0.129	0.611
No policy	23.105	24.399	0.000	0.000	0.132	0.069	1.143
No projection	39.398	0.000	0.000	0.792	0.106	0.117	0.670
SENTRY	40.190	0.000	0.000	0.000	0.106	0.120	0.657

Table 6. Mean fairness-aware candidate-selection latency on the natural workload. Feature construction and model inference are excluded from this isolated shortlist microbenchmark.

Candidate setting	Mean latency (ms)
$k = 128$	4.182
$k = 256$	4.724
$k = 512$	5.754
$k = 1024$	8.100
$k = 2048$	12.517
Full candidate set	72.930

migration difference is an independent decision-count comparison.

5.6 Robustness across policy overlays

The integrated policy–temporal–projection scheduler is evaluated across 30 deterministic policy overlays. Together, these runs contain 462,480 decisions. Every selected destination satisfies the policy, temporal-risk, and Q95 conditions. No destination conflict, migration into another active source, fallback, or missing decision is observed.

For the natural workload, the mean migration rate is 35.233%, with a 5th–95th percentile range of 30.150–44.803%. The mean next-state unsafe rate remains 0.0496%. For the service-balanced workload, the corresponding values are 37.837% and 0.1056%. Variation in migration reflects changes in the policy-admissible sets produced by each overlay; the event sequence, temporal model, risk threshold, and guardrail parameters remain fixed.

5.7 Online decision latency

Table 9 reports the latency of the complete in-memory online core. The measurement includes temporal feature construction, all required model inference, policy filtering,

fairness-aware shortlisting, ranking, reservation updates, and pre-commit verification.

The natural workload requires more computation because it contains more events per timestamp and produces a larger policy-constrained inference superset. Even its maximum observed online-core latency remains small relative to the 30-second control interval. The online feature implementation is also checked against the stored offline snapshots. The maximum feature difference is 2.22×10^{-16} , the maximum probability difference is zero, and no online placement decision differs from its stored replay reference.

5.8 Destination-capacity sensitivity

The main scheduler permits one new destination reservation per node and timestamp. Repeating the complete replay with capacity $C = 2$ does not change any migration indicator. For the service-balanced workload, all 3,787 destinations remain identical to the $C = 1$ decisions. For the natural workload, 11,617 of 11,629 selected nodes remain identical, corresponding to a destination agreement of 99.897%. The 12 changed destinations do not change the migration pattern, destination fairness, or any admission outcome. No node receives two assignments at the same timestamp under the $C = 2$ setting. The observed event density therefore does not make the default capacity-one constraint an active placement bottleneck.

6 Related Work

Admission and placement. Kubernetes already separates candidate filtering from scoring and provides reservation and pre-bind extension points [4]. SENTRY builds on this separation conceptually; its distinct question is how temporal node risk and source retention interact under a fixed policy mask. Pallevatta and Babar describe policy-as-code management of edge–cloud microservices [7]. Thijsman et al. study remotely attested Kubernetes workers [11]. Such mechanisms can supply policy and trust

Table 7. Selected paired timestamp-cluster bootstrap comparisons. Intervals are 95% percentile confidence intervals based on 20,000 bootstrap samples. Values are proportions.

Workload	Baseline and metric	Difference	CI low	CI high
Natural	Current-HGB: migration	-0.037320	-0.040806	-0.033591
Natural	Current-HGB: temporal violation	-0.003698	-0.005049	-0.002499
Natural	Current-HGB: next-state unsafe	-0.000258	-0.000574	0.000000
Natural	Policy-Q95: migration	0.009201	0.007527	0.010939
Natural	Policy-Q95: temporal violation	-0.009803	-0.011684	-0.008022
Natural	Policy-Q95: next-state unsafe	-0.001118	-0.001751	-0.000571
Natural	No projection: migration	0.009287	0.007350	0.011482
Natural	No projection: Q95 violation	-0.011437	-0.013933	-0.009146
Balanced	Current-HGB: migration	-0.030103	-0.035847	-0.024455
Balanced	Current-HGB: temporal violation	-0.005017	-0.007339	-0.002936
Balanced	Policy-Q95: migration	0.013995	0.010437	0.017647
Balanced	Policy-Q95: temporal violation	-0.013995	-0.017647	-0.010437
Balanced	Policy-Q95: next-state unsafe	-0.001848	-0.003484	-0.000528
Balanced	No projection: migration	0.007922	0.005424	0.010571
Balanced	No projection: Q95 violation	-0.007922	-0.010571	-0.005424

Table 8. Robustness across 30 deterministic policy overlays. Rates are percentages except for Jain fairness.

Workload	Migr. mean	Migr. P05	Migr. P95	Unsafe mean	Jain mean	Jain P05–P95
Natural	35.233	30.150	44.803	0.0496	0.3155	0.2722–0.3948
Service-balanced	37.837	34.927	41.211	0.1056	0.1126	0.1039–0.1226

Table 9. Complete online-core latency in milliseconds.

Statistic	Natural	Service-balanced
Mean	20.247	13.152
Median	17.710	12.990
P95	32.081	16.229
P99	57.513	17.431
Maximum	60.293	20.247

decisions. A synthetic Boolean attribute in our replay does not substitute for their enforcement or attestation work.

Resource and performance management. NetMARKS uses service-mesh communication metrics for placement [12]. KaiS studies collaborative scheduling in Kubernetes-oriented edge–cloud networks [10]. These systems motivate richer placement objectives than node utilization alone. Our evaluation does not model communication costs, so it cannot establish an advantage over them on their objectives.

FIRM addresses SLO-oriented resource management [9]; Sage studies performance debugging [2]; and Sinan uses data to manage microservice resources under QoS constraints [13]. Temporal information is therefore not itself a novel resource-management technique. Here it predicts a specific node-utilization event and controls retention or

destination admission. That target is more limited than end-to-end QoS.

Fairness and empirical margins. Madej et al. examine priority-based fair edge scheduling [6]. We instead count destination selections and report their spread using Jain’s index [3]. This does not imply fairness in useful work or user service quality. Likewise, the Q95 margin is a simple empirical persistence-residual rule, not a new uncertainty estimator. The contribution evaluated here is the behavior of these components together, with ablations that expose their costs and limits.

7 Limits and Deployment Implications

What the results establish. The replay separates three kinds of evidence. Zero policy and headroom violations show that selected nodes obey the corresponding checks. Model metrics measure prediction on held-out recorded transitions. Migration counts and the next-state proxy describe how those checks change decisions. The first is largely an invariant of filtering; it should not be read as independent evidence of a model’s accuracy or production safety.

The strongest reported comparative result is lower migration frequency than the current-state learned control. The natural-workload next-state difference against that control

has a confidence interval reaching zero. Adding Q95 to the temporal scheduler removes violations of the Q95 rule but does not improve the reported next-state proxy. Its measured benefit is compliance with that additional rule, at the cost of more migration decisions. Neither finding supports an unconditional claim of superiority.

Missing feedback and resource accounting. A selected destination does not receive additional load in this replay, and the source is not relieved of it. Repeated decisions therefore do not accumulate the physical effects that a live controller would produce. Instance signals trigger events but are not translated into destination CPU or memory demand. Before deployment, a controller needs resource requests, capacity reservations, migration costs, and a model of transient resource use. The current headroom rule cannot replace that accounting.

Scope of generalization. The evaluation uses one hour-long microservice shard from one provider, with a test interval of 59 timestamps. Synthetic policy overlays probe variation within one generator; they do not reproduce real geographic, trust, or service-sensitivity distributions. The balanced workload shares the same telemetry as the natural workload. The reported bootstrap also does not retain serial dependence between timestamps. Additional temporal windows and a block-bootstrap sensitivity analysis would strengthen the uncertainty assessment.

Destination fairness depends on both the number of migrations and the eligible sets. Jain values across workloads with different event counts are not directly comparable. History H counts retained sources as well as migrations, whereas the reported fairness metric counts migrations only. Neither captures tenant entitlement or heterogeneous node capacity.

A possible controller integration. A Kubernetes implementation could use a Filter plugin for policy and admission predicates, shared state for telemetry and history, and Reserve/Unreserve plus a final pre-bind check for coordinated decisions [4]. Reproducing the shortlist’s lexicographic selection requires explicit coordination with scoring; an ordinary weighted Score plugin does not by itself implement this ordering. Moving existing instances also requires a separate reconciliation and migration mechanism. This is a proposed mapping, not an implemented or measured integration.

Policy revocation must invalidate cached masks. Stale telemetry, model drift, controller restarts, and concurrent schedulers require explicit handling. A failed recheck must leave the event uncommitted and release any tentative reservation. Testing these paths, measuring full control-plane latency, and evaluating post-placement application behavior remain necessary before making deployment claims.

8 Conclusion

SENTRY combines hard policy admission with temporal node-risk screening, destination-use history, and an empirical headroom check. Its trace replay shows fewer migration decisions than a current-state learned control and less destination concentration than least-loaded selection. The procedure obeys its configured checks across the reported policy overlays at low in-memory computational cost. The evidence supports this decision-level result; establishing post-migration resource safety and application benefit requires an executable artifact and an evaluation that includes placement feedback.

AI assistance disclosure. OpenAI Codex assisted with restructuring and editing the manuscript, checking internal consistency, and preparing its LaTeX layout. This assistance did not execute the scheduling experiments or independently validate their reported measurements.

References

- [1] Alibaba. 2021. Alibaba Cluster Trace: Microservices v2021. <https://github.com/alibaba/clusterdata/tree/master/cluster-trace-microservices-v2021> Accessed September 19, 2026.
- [2] Yu Gan, Mingyu Liang, Sundar Dev, David Lo, and Christina Delimitrou. 2021. Sage: Practical and Scalable ML-Driven Performance Debugging in Microservices. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, 135–151.
- [3] Raj Jain, Dah-Ming Chiu, and William Hawe. 1984. A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems. *DEC Research Report TR-301* (1984).
- [4] Kubernetes Authors. 2026. Scheduling Framework. <https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/> Accessed September 19, 2026.
- [5] Shutian Luo, Huanle Xu, Chengzhi Lu, Kejiang Ye, Guoyao Xu, Liping Zhang, Yu Ding, Jian He, and Chengzhong Xu. 2021. Characterizing Microservice Dependency and Performance: Alibaba Trace Analysis. In *Proceedings of the ACM Symposium on Cloud Computing*. ACM, 412–426. <https://doi.org/10.1145/3472883.3487003>
- [6] Arkadiusz Madej, Nan Wang, Nikolaos Athanasopoulos, Rajiv Ranjan, and Blessen Varghese. 2020. Priority-Based Fair Scheduling in Edge Computing. In *2020 IEEE International Conference on Fog and Edge Computing (ICFEC)*. IEEE, 39–48. <https://doi.org/10.1109/ICFEC50348.2020.00011>
- [7] Samodha Pallewatta and Muhammad Ali Babar. 2024. Towards Secure Management of Edge-Cloud IoT Microservices Using Policy as Code.

- arXiv:2406.18813 [cs.SE] Preprint.
- [8] Fabian Pedregosa, Gael Varoquaux, Alexandre Gramfort, et al. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
 - [9] Haoran Qiu, Subho S. Banerjee, Saurabh Jha, Zbigniew T. Kalbarczyk, and Ravishankar K. Iyer. 2020. FIRM: An Intelligent Fine-Grained Resource Management Framework for SLO-Oriented Microservices. In *14th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 805–825.
 - [10] Shihao Shen, Yiwen Han, Xiaofei Wang, Shiqiang Wang, and Victor C. M. Leung. 2023. Collaborative Learning-Based Scheduling for Kubernetes-Oriented Edge–Cloud Network. *IEEE/ACM Transactions on Networking* (2023). <https://doi.org/10.1109/TNET.2023.3267168>
 - [11] Jordi Thijsman, Merlijn Sebrechts, Filip De Turck, and Bruno Volckaert. 2024. Trusting the Cloud-Native Edge: Remotely Attested Kubernetes Workers. arXiv:2405.10131
 - [12] Lukasz Wojciechowski, Krzysztof Opasiak, Jacek Latusek, Maciej Wereski, Victor Morales, Taesang Kim, and Minhong Hong. 2021. NetMARKS: Network Metrics-Aware Kubernetes Scheduler Powered by Service Mesh. In *IEEE INFOCOM*. IEEE, 1–9.
 - [13] Yanqi Zhang, Weizhe Hua, Zhuangzhuang Zhou, G. Edward Suh, and Christina Delimitrou. 2021. Sinan: Data-Driven, QoS-Aware Cluster Management for Microservices. <https://doi.org/10.48550/arXiv.2105.13424> arXiv:2105.13424 [cs.DC]